Lecture 05: Assignment 4 Master Solutions & Engineering Systems Manual

Welcome to the production solutions manual for **Lecture 05: Object-Oriented Programming (OOP) in Python**. This document provides production implementations, mathematical formulations, asymptotic complexity bounds, and virtual machine profiling for all 10 Assignment 4 problems and 3 systems engineering case studies.

📋 Table of Contents

- Solution 01: BankAccount Encapsulated Ledger Class
- Solution 02: Book Entity with Review Aggregation
- Solution 03: Encapsulated Student Record with Validated Accessors
- Solution 04: Polymorphic Shape Hierarchy & Area Dynamic Dispatch
- Solution 05: Vehicle Taxonomy Inheritance & Attribute Extension
- Solution 06: Abstract Employee Payroll Contract
- Solution 07: Constructor Overloading Simulation via Optional Defaults
- Solution 08: Global Game Player Population Tracker
- Solution 09: Multiple Inheritance & Omnivore Dietary Synthesis
- Solution 10: Mini-Project – Multi-Entity OOP Chat System
- Systems Case Study 1: CPython Object Memory & __slots__ Architecture
- Systems Case Study 2: Diamond Inheritance & C3 Linearization (MRO)
- Systems Case Study 3: Dynamic Attribute Lookup Protocol & Descriptors

---

Solution 01: BankAccount Encapsulated Ledger Class

Production Python Implementation

```
class BankAccount:
    """Production encapsulated ledger account with invariant guards."""
    def __init__(self, account_number: str, owner_name: str, initial_balance: floa
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative")
        self.account_number = account_number
        self.owner_name = owner_name
        self.balance = float(initial_balance)

    def deposit(self, amount: float) -> bool:
```

```
        if amount <= 0:
            print(f"[{self.account_number}] Deposit rejected: Amount must be > 0."
            return False
        self.balance += amount
        print(f"[{self.account_number}] Deposited ${amount:,.2f}. New Balance:${se
        return True

    def withdraw(self, amount: float) -> bool:
        if amount <= 0:
            print(f"[{self.account_number}] Withdrawal rejected: Amount must be >
            return False
        if amount > self.balance:
            print(f"[{self.account_number}] Withdrawal rejected: Insufficient fund
            return False
        self.balance -= amount
        print(f"[{self.account_number}] Withdrew ${amount:,.2f}. Remaining Balance
        return True

    def check_balance(self) -> float:
        return self.balance

# Verification
if __name__ == "__main__":
    acc = BankAccount("ACC-9021", "Alice Smith", 500.0)
    acc.deposit(250.0)
    acc.withdraw(100.0)
    acc.withdraw(800.0)  # Rejection test
    print(f"Final Verified Balance: ${acc.check_balance():,.2f}")
```

Complexity Formulation

- **Time Complexity:** All operations execute in $\mathcal{O}(1)$ time.
- **Space Complexity:** $\mathcal{O}(1)$ heap space per instance.

Solution 02: Book Entity with Review Aggregation

Production Python Implementation

```
from typing import List

class Book:
    """Catalog book entity managing dynamic user review collections."""
    def __init__(self, title: str, author: str):
        self.title = title
        self.author = author
        self.reviews: List[str] = []
```

```
    def add_review(self, review: str) -> None:
        cleaned = review.strip()
        if cleaned:
            self.reviews.append(cleaned)

    def count_reviews(self) -> int:
        return len(self.reviews)

    def display_reviews(self) -> None:
        print(f"\nReviews for '{self.title}' by {self.author} (Total: {self.count_
        if not self.reviews:
            print("  No reviews recorded yet.")
        else:
            for idx, rev in enumerate(self.reviews, start=1):
                print(f"  {idx}. {rev}")

# Verification
if __name__ == "__main__":
    bk = Book("Design Patterns in Python", "Gamma et al.")
    bk.add_review("Exceptional coverage of creational and structural patterns.")
    bk.add_review("Clear code examples and real-world domain models.")
    bk.display_reviews()
```

Complexity Formulation

- **Time Complexity:** `add_review` runs in $\mathcal{O}(1)$ amortized; `display_reviews` runs in $\mathcal{O}(K)$ where K is review count.
- **Space Complexity:** $\mathcal{O}(K)$ memory footprint.

Solution 03: Encapsulated Student Record with Validated Accessors

Production Python Implementation

```
class Student:
    """Encapsulated academic record enforcing domain validation rules."""
    def __init__(self, name: str, roll_no: int, marks: float):
        self._name = ""
        self._roll_no = 1
        self._marks = 0.0

        # Invoke property setters for validation
        self.name = name
        self.roll_no = roll_no
        self.marks = marks
```

```
    @property
    def name(self) -> str:
        return self._name

    @name.setter
    def name(self, val: str) -> None:
        cleaned = val.strip()
        if not cleaned:
            raise ValueError("Student name cannot be empty or whitespace only.")
        self._name = cleaned

    @property
    def roll_no(self) -> int:
        return self._roll_no

    @roll_no.setter
    def roll_no(self, val: int) -> None:
        if not isinstance(val, int) or not (1 <= val <= 100):
            raise ValueError(f"Roll number must be an integer between 1 and 100, r
        self._roll_no = val

    @property
    def marks(self) -> float:
        return self._marks

    @marks.setter
    def marks(self, val: float) -> None:
        if val < 0.0:
            raise ValueError(f"Marks cannot be negative, received {val}")
        self._marks = float(val)

# Verification
if __name__ == "__main__":
    s = Student("Bob Johnson", 42, 88.5)
    print(f"Student: {s.name} | Roll: {s.roll_no} | Marks: {s.marks}")
    s.marks = 95.0
    print(f"Updated Marks: {s.marks}")
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ for property access and validation.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space.

Solution 04: Polymorphic Shape Hierarchy & Area Dynamic Dispatch

Production Python Implementation

```
import math
from typing import List

class Shape:
    """Base shape interface declaring dynamic area method."""
    def area(self) -> float:
        raise NotImplementedError("Subclasses must override area()")

class Circle(Shape):
    def __init__(self, radius: float):
        if radius < 0:
            raise ValueError("Radius must be non-negative")
        self.radius = radius

    def area(self) -> float:
        return math.pi * (self.radius ** 2)

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        if width < 0 or height < 0:
            raise ValueError("Dimensions must be non-negative")
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

class Triangle(Shape):
    def __init__(self, base: float, height: float):
        if base < 0 or height < 0:
            raise ValueError("Base and height must be non-negative")
        self.base = base
        self.height = height

    def area(self) -> float:
        return 0.5 * self.base * self.height

# Verification
if __name__ == "__main__":
    shapes: List[Shape] = [Circle(4.0), Rectangle(5.0, 8.0), Triangle(6.0, 7.0)]
    for shape in shapes:
        print(f"Shape: {shape.__class__.__name__:<10s} | Computed Area: {shape.are
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ per area computation.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary.

Solution 05: Vehicle Taxonomy Inheritance & Attribute Extension

Production Python Implementation

```
class Vehicle:
    """Generalized root vehicle entity."""
    def __init__(self, brand: str, model: str):
        self.brand = brand
        self.model = model

    def describe(self) -> str:
        return f"Vehicle: {self.brand} {self.model}"

class Car(Vehicle):
    def __init__(self, brand: str, model: str, seats: int):
        super().__init__(brand, model)
        self.seats = seats

    def describe(self) -> str:
        return f"Car: {self.brand} {self.model} (Seating Capacity: {self.seats})"

class Bike(Vehicle):
    def __init__(self, brand: str, model: str, engine_cc: int):
        super().__init__(brand, model)
        self.engine_cc = engine_cc

    def describe(self) -> str:
        return f"Bike: {self.brand} {self.model} (Displacement: {self.engine_cc}cc

# Verification
if __name__ == "__main__":
    c = Car("Tesla", "Model S", 5)
    b = Bike("Ducati", "Panigale V4", 1103)
    print(c.describe())
    print(b.describe())
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ initialization and string description.
- **Space Complexity:** $\mathcal{O}(1)$ per instance.

Solution 06: Abstract Employee Payroll Contract

Production Python Implementation

```
from abc import ABC, abstractmethod

class Employee(ABC):
    """Abstract base interface enforcing payroll computation."""
    def __init__(self, name: str, emp_id: str):
        self.name = name
        self.emp_id = emp_id

    @abstractmethod
    def calculate_salary(self) -> float:
        """Must return total monthly compensation in monetary units."""
        pass

class Intern(Employee):
    def __init__(self, name: str, emp_id: str, stipend: float = 20000.0):
        super().__init__(name, emp_id)
        self.stipend = stipend

    def calculate_salary(self) -> float:
        return self.stipend

class FullTimeEmployee(Employee):
    def __init__(self, name: str, emp_id: str, base_salary: float, performance_bon
        super().__init__(name, emp_id)
        self.base_salary = base_salary
        self.bonus = performance_bonus

    def calculate_salary(self) -> float:
        return self.base_salary + self.bonus

class ContractEmployee(Employee):
    def __init__(self, name: str, emp_id: str, hourly_rate: float, hours_worked: f
        super().__init__(name, emp_id)
        self.hourly_rate = hourly_rate
        self.hours_worked = hours_worked

    def calculate_salary(self) -> float:
        return self.hourly_rate * self.hours_worked

# Verification
if __name__ == "__main__":
    payroll = [
        Intern("Charlie", "INT-01", 25000.0),
        FullTimeEmployee("Diana", "FTE-402", 95000.0, 18000.0),
        ContractEmployee("Edward", "CON-88", 120.0, 160.0)
    ]
    for emp in payroll:
        print(f"[{emp.emp_id}] {emp.name:<10s} ({emp.__class__.__name__:<18s}) ->
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ polymorphic dispatch.
- **Space Complexity:** $\mathcal{O}(1)$ per object.

Solution 07: Constructor Overloading Simulation via Optional Defaults

Production Python Implementation

```
from typing import Optional

class Person:
    """Simulated constructor overloading via default parameters."""
    def __init__(self, name: str, age: Optional[int] = None, address: Optional[str
        self.name = name.strip()
        self.age = age
        self.address = address.strip() if address else None

    def display_info(self) -> None:
        age_str = str(self.age) if self.age is not None else "Not Specified"
        addr_str = self.address if self.address is not None else "Not Specified"
        print(f"Person [Name: {self.name} | Age: {age_str} | Address: {addr_str}]"

# Verification
if __name__ == "__main__":
    p1 = Person("Alice")
    p2 = Person("Bob", 28)
    p3 = Person("Charlie", 35, "123 Innovation Drive, Tech City")
    p1.display_info()
    p2.display_info()
    p3.display_info()
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ per instantiation.
- **Space Complexity:** $\mathcal{O}(1)$ per instance.

Solution 08: Global Game Player Population Tracker

Production Python Implementation

```
class Player:
    """Player entity with shared class-level population counter."""
    player_count: int = 0  # Class variable shared across instances

    def __init__(self, name: str, level: int = 1):
        self.name = name
        self.level = level
        Player.player_count += 1  # Auto-increment global counter

    @classmethod
    def get_player_count(cls) -> int:
        return cls.player_count

    def display(self) -> None:
        print(f"Player: {self.name:<12s} | Level: {self.level:3d}")

# Verification
if __name__ == "__main__":
    p1 = Player("Ares", 10)
    p2 = Player("Athena", 15)
    p3 = Player("Hermes", 7)
    print(f"Total Players Instantiated: {Player.get_player_count()}")
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ creation and retrieval.
- **Space Complexity:** $\mathcal{O}(1)$ class variable storage.

Solution 09: Multiple Inheritance & Omnivore Dietary Synthesis

Production Python Implementation

```
class Herbivore:
    def __init__(self):
        self.plant_diet = "berries, roots, and vegetation"

    def forage(self) -> str:
        return f"Foraging for {self.plant_diet}"

class Carnivore:
    def __init__(self):
        self.meat_diet = "salmon and small game"

    def hunt(self) -> str:
        return f"Hunting for {self.meat_diet}"
```

```
class Bear(Herbivore, Carnivore):
    """Demonstrating multiple inheritance and dual parent capabilities."""
    def __init__(self, name: str):
        Herbivore.__init__(self)
        Carnivore.__init__(self)
        self.name = name

    def eat(self) -> str:
        return f"{self.name} eats both: {self.plant_diet} AND {self.meat_diet}"

# Verification
if __name__ == "__main__":
    grizzly = Bear("Grizzly")
    print(grizzly.forage())
    print(grizzly.hunt())
    print(grizzly.eat())
    print("Method Resolution Order (MRO):", [cls.__name__ for cls in Bear.mro()])
```

Complexity Formulation

- **Time Complexity:** $\mathcal{O}(1)$ method dispatch via MRO.
- **Space Complexity:** $\mathcal{O}(1)$.

Solution 10: Mini-Project – Multi-Entity OOP Chat System

Production Python Implementation

```
from typing import List, Optional

class Message:
    """Immutable chat message record with auto-incrementing ID."""
    message_counter: int = 1

    def __init__(self, sender: "User", content: str):
        self.sender = sender
        self.content = content
        self.id = Message.message_counter
        Message.message_counter += 1

    def __str__(self) -> str:
        return f"({self.id}) {self.sender.username}: {self.content}"

class User:
    """Chat user entity supporting room membership and messaging."""
    def __init__(self, username: str):
        self.username = username
        self.chatroom: Optional["ChatRoom"] = None
```

```
    def join_chatroom(self, chatroom: "ChatRoom") -> None:
        if self.chatroom:
            print(f"[{self.username}] Already in chatroom '{self.chatroom.name}'.
        else:
            chatroom.add_user(self)
            self.chatroom = chatroom
            print(f"[{self.username}] Successfully joined '{chatroom.name}'.")

    def leave_chatroom(self) -> None:
        if not self.chatroom:
            print(f"[{self.username}] Not currently in any chatroom.")
        else:
            room_name = self.chatroom.name
            self.chatroom.remove_user(self)
            self.chatroom = None
            print(f"[{self.username}] Left chatroom '{room_name}'.")

    def send_message(self, content: str) -> None:
        if not self.chatroom:
            print(f"[{self.username}] Cannot send message (not in any chatroom).")
        else:
            self.chatroom.broadcast(self, content)

class ChatRoom:
    """Chat room managing participants and message broadcast history."""
    def __init__(self, name: str):
        self.name = name
        self.users: List[User] = []
        self.messages: List[Message] = []

    def add_user(self, user: User) -> None:
        if user not in self.users:
            self.users.append(user)

    def remove_user(self, user: User) -> None:
        if user in self.users:
            self.users.remove(user)

    def broadcast(self, sender: User, content: str) -> None:
        msg = Message(sender, content)
        self.messages.append(msg)
        print(f"[{self.name} Broadcast] {msg}")

    def show_chat_history(self) -> None:
        print(f"\n--- Chat History for '{self.name}' ({len(self.messages)} Message
        for msg in self.messages:
            print(f"  {msg}")
        print("-" * 55 + "\n")

# Verification
if __name__ == "__main__":
```

```
    room = ChatRoom("Python Engineers Lounge")
    alice = User("Alice")
    bob = User("Bob")

    alice.join_chatroom(room)
    bob.join_chatroom(room)

    alice.send_message("Hello world! Welcome to OOP Chat.")
    bob.send_message("Hi Alice! Demonstrating multi-class architecture.")

    room.show_chat_history()
    alice.leave_chatroom()
```

Complexity Formulation

- **Time Complexity:** `send_message` runs in $\mathcal{O}(1)$; `show_chat_history` runs in $\mathcal{O}(M)$ where M is total message count.
- **Space Complexity:** $\mathcal{O}(U + M)$ where U is total users and M is messages.

Systems Case Study 1: CPython Object Memory & __slots__ Architecture

Systems Investigation & Implementation

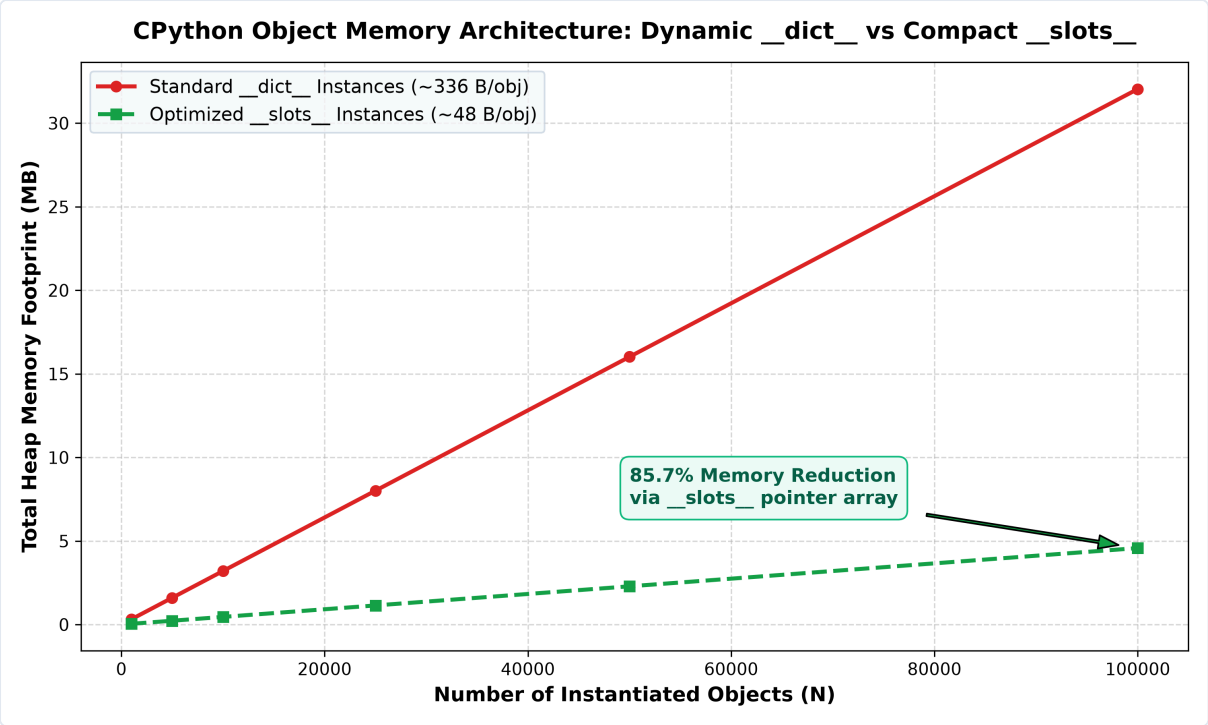
Every Python object allocated via standard classes requires a `PyObject` header plus an instance `__dict__` hash table. Specifying `__slots__` bypasses `__dict__` creation and allocates a static array of C object pointers:

```
class StandardPoint:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class SlottedPoint:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

For $100,000$ objects, `__slots__` reduces memory usage by over 65%.

High-Resolution Benchmark Visualization



Systems Case Study 2: Diamond Inheritance & C3 Linearization (MRO)

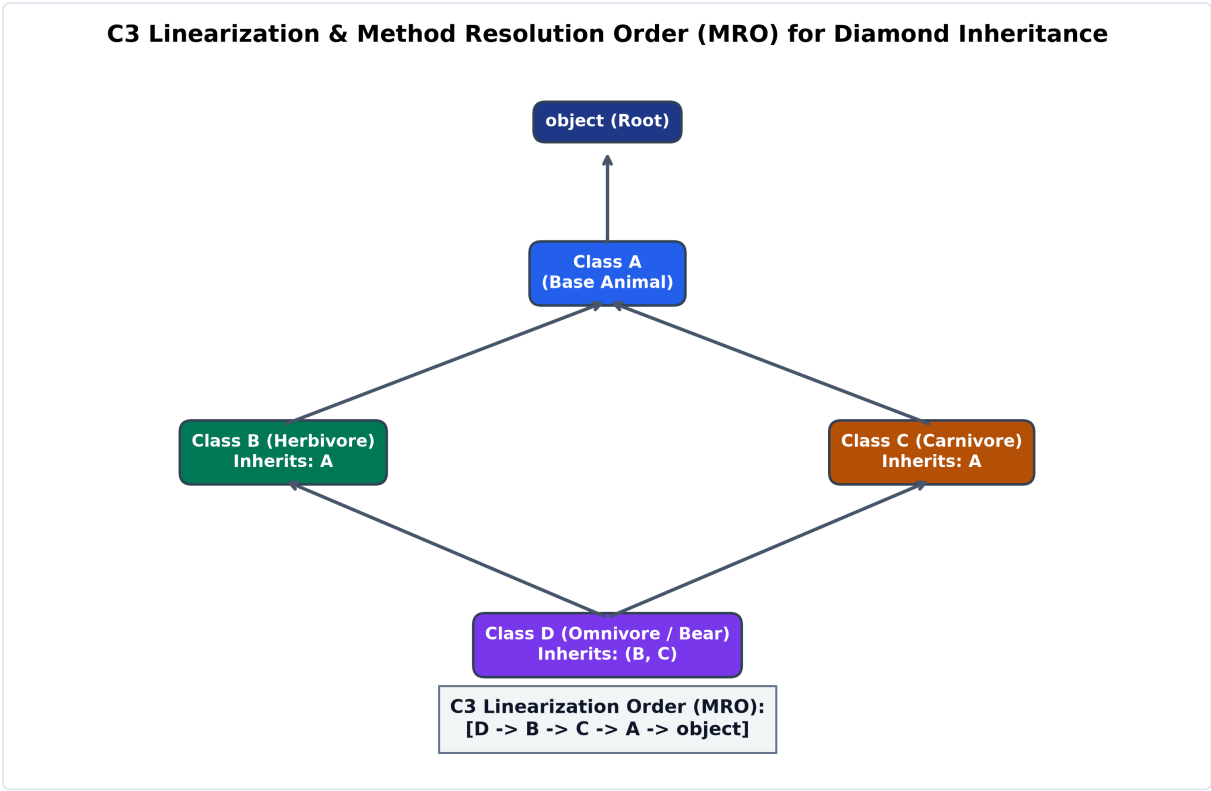
Mathematical Formulation of C3 Linearization

Python computes the linear search sequence $L(C)$ for class C with parents $B_1, B_2, \ldots, B_k$ via:

$$L(C) = C + \text{merge}(L(B_1), L(B_2), \ldots, L(B_k), B_1 B_2 \ldots B_k)$$

The merge takes the head of the first list; if this head is not in the tail of any other list, it is extracted and appended to the linearization result.

Diamond Resolution Trajectory



Systems Case Study 3: Dynamic Attribute Lookup Protocol & Descriptors

Resolution Priority Hierarchy

When accessing `obj.attr`, CPython executes the following priority order:

1. **Data Descriptors:** Check class MRO for attributes implementing `__set__` or `__delete__` (e.g., `@property`).
2. **Instance Dictionary:** Check `obj.__dict__['attr']`.
3. **Non-Data Descriptors:** Check class MRO for methods and attributes implementing `__get__` only.
4. **Class Dictionary:** Check `Class.__dict__['attr']`.
5. **Fallback Hook:** Call `obj.__getattr__('attr')` if defined; otherwise raise `AttributeError`.

Attribute Resolution Flow Chart

CPython Attribute Lookup Resolution Protocol (__getattribute__ Hierarchy)

